

Obliczenia Naukowe, laboratorium, Lista 1

Błażej Pawluk

25 października 2025

1 Zadanie 1

1.1 Opis problemu:

Epsilon maszynowy (*macheps*), czyli liczba *macheps* taka, że $macheps > 0$, $fl(1.0 + macheps) > 1.0$, $fl(1.0 + macheps) = 1 + macheps$.

Liczba maszynowa (*eta*), czyli liczba *eta* taka, że $eta > 0.0$ - najmniejsza liczba większa od 0 możliwa do wyrażenia w danej arytmetyce.

MAX, czyli największa liczba możliwa do wyrażenia w danej arytmetyce.

Inne zagadnienia: związek między liczbą *macheps* a precyzją arytmetyki, liczbą *eta* a MIN_{sub} , wynikiem funkcji `floatmin(type)` a MIN_{nor} .

1.2 Rozwiązanie:

Epsilon maszynowy: iteracyjnie znaleziona liczba *macheps* dla typów `half`, `single`, `double` poprzez podnoszenie liczby 2 do kolejnych ujemnych potęg dopóki spełniała podany warunek.

Liczba maszynowa: iteracyjnie znaleziona liczba *eta* dla typów `half`, `single`, `double` w analogiczny sposób jak dla liczby *macheps*.

MAX: iteracyjnie znaleziona liczba *MAX* dla typów `half`, `single`, `double` poprzez podnoszenie liczby 2 do coraz większej potęgi, dopóki nie jest rozpoznawana jako nieskończoność.

1.3 Wyniki i ich interpretacja:

Epsilon maszynowy:

arytmetyka	Float16	Float32	Float64
wynik iteracyjny	0.000977	1.1920929e-7	2.220446049250313e-16
<code>eps(type)</code>	0.000977	1.1920929e-7	2.220446049250313e-16
biblioteka <code>float.h</code>	brak	1.1920929e-7	2.220446049250313e-16

Tabela 1: Porównanie wartości epsilonu maszynowego wyliczonego iteracyjnie, z funkcji `eps(type)` oraz wartości z biblioteki `float.h`.

Liczba maszynowa:

arytmetyka	Float16	Float32	Float64
wynik iteracyjny	6.0e-8	1.0e-45	5.0e-324
<code>nextfloat(type(0.0))</code>	6.0e-8	1.0e-45	5.0e-324

Tabela 2: Porównanie wartości liczby maszynowej wyliczonej iteracyjnie oraz z funkcji `nextfloat(type(0.0))`.

Liczba MAX:

arytmetyka	Float16	Float32	Float64
wynik iteracyjny	6.55e4	3.4028235e38	1.7976931348623157e308
<code>floatmax(type)</code>	6.55e4	3.4028235e38	1.7976931348623157e308
biblioteka <code>float.h</code>	brak	3.4028235e38	1.7976931348623157e308

Tabela 3: Porównanie wartości liczby *MAX* wyliczonej iteracyjnie, z funkcji `floatmax(type)` oraz wartości z biblioteki `float.h`.

Wyniki funkcji `floatmin(type)`:

arytmetyka	Float16	Float32	Float64
<code>floatmin(type)</code>	6.104e-5	1.1754944e-38	2.2250738585072014e-308

Tabela 4: Wartości funkcji `floatmin(type)` dla różnych arytmetyk.

Wyniki pomimo różnych metod są takie same.

1.4 Wnioski:

Epsilon maszynowy: Liczba *macheps* przyjmuje wartość 2^{-t} , gdzie t to liczba cyfr mantysy (dla `half` jest to 10 cyfr, więc $macheps = 2^{-10} = 0.000977$ i analogicznie dla kolejnych typów). Na wykładzie precyzję arytmetyki wyliczaliśmy ze wzoru $0.5 * \beta^{1-t}$, gdzie β jest bazą rozwinięcia (dla tego przykładu $\beta = 2$), a t to liczba cyfr mantysy. $0.5 * \beta^{1-t} = 2^{-1} * 2^{1-t} = 2^{-t}$ zatem epsilon maszynowy jest równoznaczny z precyzją arytmetyki.

Liczba maszynowa: Liczba *eta* przyjmuje wartość 2^{-t+c} , gdzie c to minimalna cecha arytmetyki (dla `half` jest to $-14 = -2^5 + 2$ (5 bitowa cecha), zatem $eta = 2^{-10-14} = 2^{-24} = 6.0 * 10^{-8}$). Na wykładzie MIN_{sub} wyliczaliśmy ze wzoru $2^{-(t-1)} * 2^{c_{min}}$, gdzie t to liczba cyfr mantysy, c_{min} - minimalna cecha. Zatem $MIN_{sub} = 2^{-t+c_{min}+1} = 2 * eta$, zatem *eta* jest 2 razy mniejsza od MIN_{sub} .

`floatmin(type)` a MIN_{nor} : Wartości zwracane przez funkcję `floatmin(type)` są równe z wartościami MIN_{nor} podanymi na wykładzie i zgodne ze wzorem $2^{c_{min}}$.

Liczba MAX: Liczba *MAX* przyjmuje wartość $2^{2^d-1} * (1 + (1 - 2^{-t}))$, czyli maksymalna wartość cechy niepowodująca nieskończoności lub NaN (11..10) oraz maksymalna wartość mantysy (11..1). Wartości te pokrywają się z wartościami i wzorami podanymi na wykładzie.

2 Zadanie 2

2.1 Opis problemu:

Teza: epsilon maszynowy (*macheps*) można otrzymać poprzez obliczenie wyrażenia $3(4/3 - 1) - 1$ w arytmetyce zmiennopozycyjnej.

2.2 Rozwiązanie:

Obliczenie podanego wyrażenia w różnych arytmetykach (*half*, *single*, *double*).

2.3 Wyniki i ich interpretacja:

arytmetyka	Float16	Float32	Float64
wynik wyrażenia	-0.000977	1.1920929e-7	-2.220446049250313e-16
wartość dokładna	0.000977	1.1920929e-7	2.220446049250313e-16

Tabela 5: Porównanie wyniku wyrażenia $3(4/3 - 1) - 1$ i wartości *macheps*.

Wartości się zgadzają co do wartości bezwzględnej.

2.4 Wnioski:

Wyrażenie $3(4/3 - 1) - 1$ zwraca $\pm macheps$ w zależności od parzystości rozmiaru cechy. Dla *half* oraz *double* wyrażenie zwraca $-macheps$, ponieważ rozmiary cechy są nieparzyste (dla *half* 5, a dla *double* 11), natomiast dla *single* wyrażenie wynosi *macheps*, ponieważ rozmiar cechy jest parzysty (8).

3 Zadanie 3

3.1 Opis problemu:

Sprawdzenie rozmieszczenia liczb zmiennoprzecinkowych w danych przedziałach. **Teza:** w przedziale $[1, 2]$ są one rozmieszczone z krokiem $\delta = 2^{-52}$.

3.2 Rozwiązanie:

Do 2 liczb z podanego przedziału dodaję δ lub $\delta/2$. Otrzymany wynik porównuję z pierwotną liczbą oraz sprawdzam jak wygląda jako ciąg bitów przy pomocy funkcji *bitstring*.

3.3 Wyniki i interpretacja:

Przedział $[1, 2]$: $\delta = 2^{-52}$

$$fl(1.0 + 2^{-53}) = 1.0:$$

```
0|0111111111|0000000000000000000000000000000000000000000000000000000
```

$$fl(1.0 + 2^{-52}) = 1.00000000000000002:$$
[illegible]
$$fl(1.5 + 2^{-53}) = 1.5:$$

`0|0111111111|100000000000000000000000000000000000000000000000000`

$$fl(1.5 + 2^{-52}) = 1.50000000000000002:$$
[illegible]

Liczba $\delta/2 = 2^{-53}$ jest zbyt mała, aby zaokrąglenie jej nie pominięło, natomiast dodanie $\delta = 2^{-52}$ już powoduje zmianę wyniku i otrzymanie innego wyniku niż liczba początkowa. Widać to też po wyniku funkcji `bitstring`, gdzie można zauważyć, że po dodaniu *delta* zmienia się jedynie najmniej znaczący bit mantysy, co oznacza, że nie ma liczby pomiędzy x i $x + \delta$, którą można wyróżnić w arytmetyce `double`.

Przedział $[0.5, 1]$: $\delta = 2^{-53}$

$$fl(0.5 + 2^{-54}) = 0.5:$$

$\text{0} \mid 0111111110 \mid 000$

$$fl(0.5 + 2^{-53}) = 0.50000000000000001:$$
[illegible]
$$fl(0.75 + 2^{-54}) = 0.75:$$

`0|0111111110|100000000000000000000000000000000000000000000000000`

$$fl(0.75 + 2^{-53}) = 0.750000000000000001:$$
[illegible]

Wyniki są podobne do przedziału $[1, 2]$, z różnicą wartości δ równej dla przedziału $[0.5, 1]$ 2^{-53} .

Przedział $[2, 4]$: $\delta = 2^{-51}$

$$fl(2.0 + 2^{-52}) = 2.0:$$

0 | 10000000000 | 000

$$fl(2.0 + 2^{-51}) = 2.00000000000000004:$$
[illegible]
$$fl(3.0 + 2^{-52}) = 3.0:$$

0 | 10000000000 | 100

$$fl(3.0 + 2^{-51}) = 3.000000000000000004:$$
[illegible]

Wyniki są podobne do przedziału $[1, 2]$, z różnicą wartości *delta* równej dla przedziału $[2, 4]$ 2^{-51} .

3.4 Wnioski:

W systemie `double` mantysa ma 52 bity, zatem najmniej znaczący bit ma wartość 2^{-52} . Cecha, która decyduje do jakiego przedziału należy liczba sprawia, że część ułamkowa może mieć mniejszą lub większą wartość - dla przedziału $[1, 2]$ cecha wynosi 0, zatem najmniejszą liczbę $x : x > 1.0$ można wyrazić jako $x = 2^0 * (1 + 2^{-52})$, co oznacza, że w tym przedziale $\delta = 2^{-52}$. Analogicznie dla przedziału $[0.5, 1]$ cecha wynosi -1 , więc $x = 2^{-1} * (1 + 2^{-52}) = 0.5 + 2^{-53}$, zatem $\delta = 2^{-53}$.

4 Zadanie 4

4.1 Opis problemu:

Znalezienie najmniejszej liczby x takiej, że $1 < x < 2$ oraz $fl(x * (1/x)) \neq 1$.

4.2 Rozwiązanie:

Sprawdzanie podanego warunku po kolei dla wszystkich liczb typu `Float64` z przedziału $(1, 2)$, od najmniejszej do momentu znalezienia liczby spełniającej dany warunek.

4.3 Wyniki i ich interpretacja:

$x = 1.000000057228997$
 $x * (1/x) = 0.9999999999999999$

4.4 Wnioski:

Zaokrąglanie niezbędne do działania arytmetyki `double` sprawia, że oczywiste dla człowieka działania matematyczne, jeżeli wpisane do komputera bez odpowiedniej analizy mogą zwracać błędne wyniki.

5 Zadanie 5

5.1 Opis problemu:

Obliczanie iloczynu skalarnego dwóch wektorów na różne sposoby:
 $x = [2.718281828, -3.141592654, 1.414213562, 0.5772156649, 0.3010299957]$,
 $y = [1486.2497, 878366.9879, -22.37492, 4773714.647, 0.000185049]$.

5.2 Rozwiązanie:

Napisanie algorytmu obliczającego iloczyn skalarny 2 podanych wektorów na 4 sposoby (w przód, w tył, od największego (co do wartości bezwzględnej) oraz analogicznie od najmniejszego) w 2 różnych arytmetykach (`single` i `double`).

5.3 Wyniki i ich interpretacja:

arytmetyka	Float32	Float64
w przód	-0.4999443	1.0251881368296672e-10
w tył	-0.4543457	-1.5643308870494366e-10
od największego	-0.5	0.0
od najmniejszego	-0.5	0.0
wartość dokładna	-1.00657107000000e-11	-1.00657107000000e-11

Tabela 6: Porównanie wyliczonych w różne sposoby i w różnych arytmetykach wyników iloczynu skalarnego wektorów x i y z dokładną wartością tego iloczynu.

Wartości dla wszystkich sposobów obliczenia bardzo odbiegają od dokładnego wyniku.

5.4 Wnioski:

Zaokrąglenia w arytmetykach powodują pominięcie rozwinięć, które wpływają na ostateczny wynik. Aby wynik był dokładniejszy należy zwiększać rozmiar mantysy oraz dobrać sposób obliczania dobry do danego problemu i danych (sposób pozwalający na uzyskanie lepszej dokładności np. w problemie sumowania kolejnych liczb - posortowanie liczb według rozmiaru przed ich dodaniem, nie koniecznie musi się sprawdzić w innym problemie, co pokazują uzyskane wyniki).

6 Zadanie 6

6.1 Opis problemu:

Obliczenie wartości dwóch funkcji:

$$f(x) = \sqrt{x^2 + 1} - 1 \text{ oraz } g(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}$$

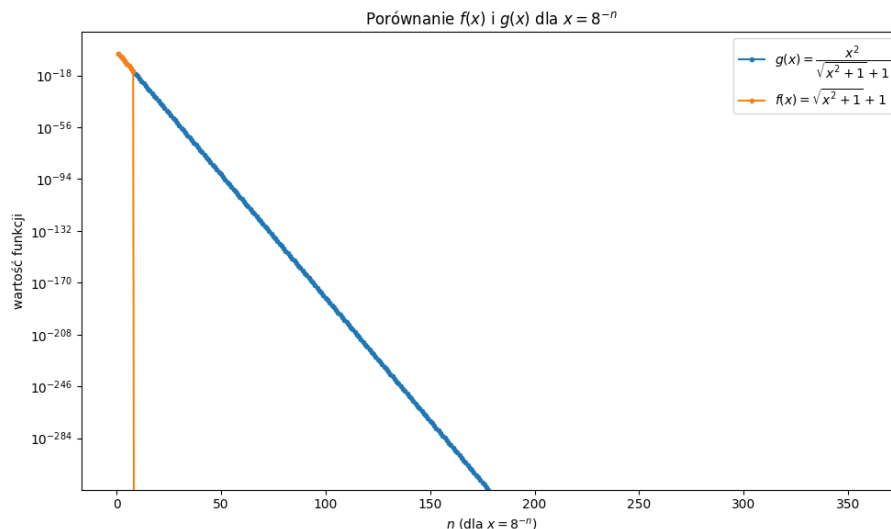
dla kolejnych wartości $x = 8^{-1}, 8^{-2}, \dots$

6.2 Rozwiązanie:

Napisanie programu wyliczającego interesujące nas dane i przedstawienie ich na wykresie.

6.3 Wyniki i ich interpretacja:

Funkcja $f(x)$ szybko się zeruje - już po 8 iteracjach, natomiast funkcja $g(x)$ dopiero po 178. Dopóki obie funkcje zwracają wartość większą od 0, to ich wartości są bardzo zbliżone.



Rysunek 1: Wartości funkcji $f(x)$ oraz $g(x)$

6.4 Wnioski:

Zgodnie z zasadami działania arytmetyki `double` w funkcji $f(x)$ wartość x jest szybko ignorowana przez zaokrąglanie przy dodawaniu oraz pierwiastkowaniu, przez co funkcja zwraca wartość 0 już po 8 iteracjach. Natomiast w funkcji $g(x)$ wartość x w mianowniku jest ignorowana w tym samym momencie co w funkcji $f(x)$, jednak wartość x z licznika przy dzieleniu nie jest pomijana przez dłuższy czas.

Dopóki funkcja $f(x)$ nie pomija wartości x , to zwraca dokładniejsze wyniki z powodu mniejszej liczby operacji, a co za tym idzie - mniejszego błędu zaokrąglania. Jednak dla późniejszych iteracji lepsze wyniki zwraca funkcja $g(x)$, ponieważ x ma dłużej wpływ na wynik.

7 Zadanie 7

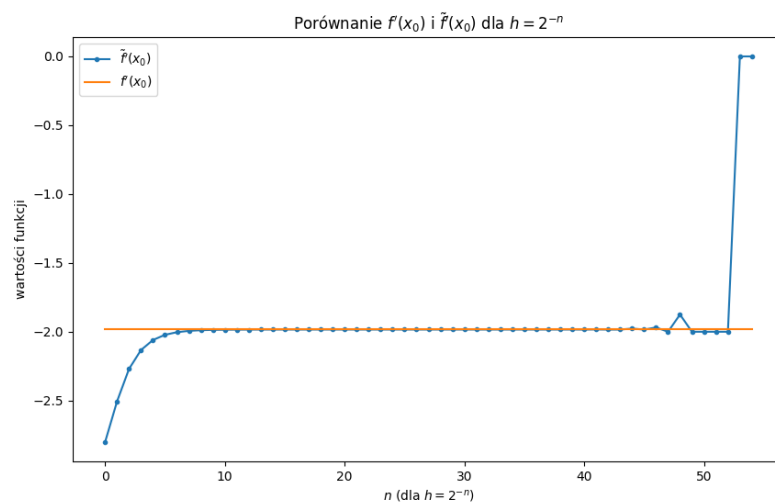
7.1 Opis problemu:

Obliczenie pochodnej funkcji $f(x) = \sin x + \cos 3x$ w punkcie $x_0 = 1$ przy pomocy wzoru $f'(x_0) \approx \tilde{f}'(x_0) = \frac{f(x_0+h) - f(x_0)}{h}$ oraz błędów przybliżenia w porównaniu do dokładnej wartości dla $h = 2^{-n}$ ($n = 0, 1, \dots, 54$).

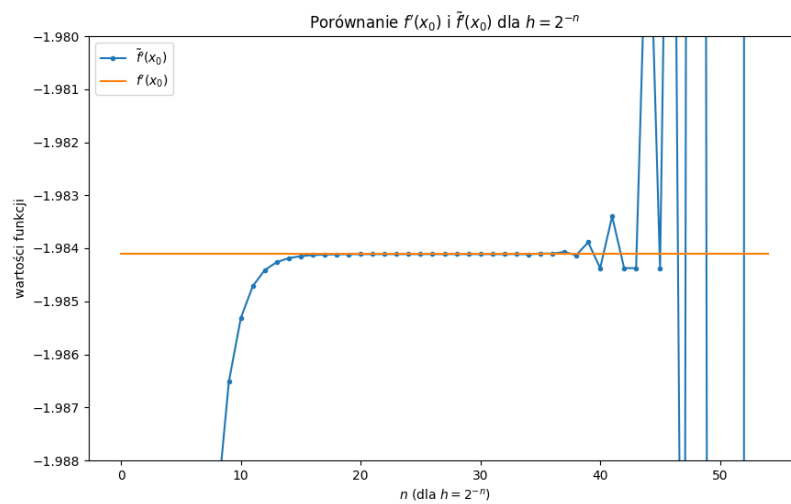
7.2 Rozwiązanie:

Napisanie programu wykonujący odpowiednie obliczenia w arytmetyce `double` i przedstawienie wyników na wykresie.

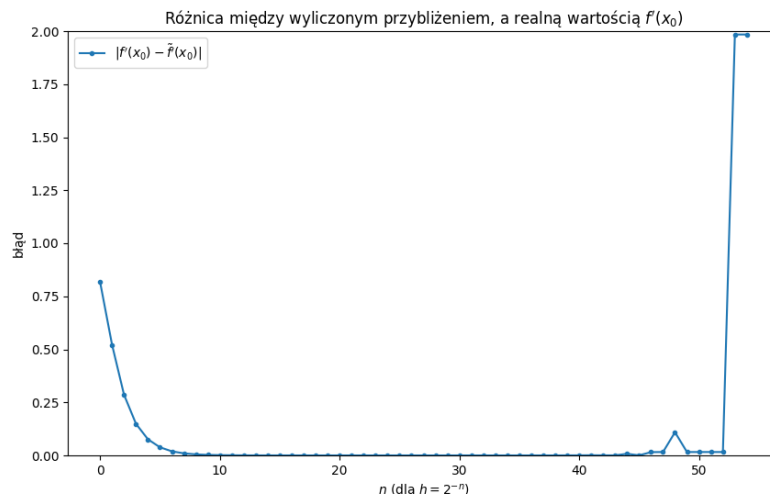
7.3 Wyniki i interpretacja:



Rysunek 2: Porównanie przybliżenia i dokładnej wartości pochodnej $f'(x_0)$



Rysunek 3: Przybliżony wykres porównania przybliżenia i dokładnej wartości pochodnej $f'(x_0)$



Rysunek 4: Błędy dla kolejnych wartości $h = 2^{-n}$

Już dla $n \approx 10$ przybliżenie jest bardzo zbliżone do realnej wartości pochodnej, natomiast dla wartości $n > 35$ przybliżenie zaczyna ponownie odbiegać od realnej wartości. Najbliższe przybliżenie jest dla wartości 25 i 26.

7.4 Wnioski:

Dopóki arytmetyka `double` nie wymusza zaokrągleń przybliżenie, zgodnie ze wzorem jest coraz dokładniejsze. Natomiast gdy niezbędne są zaokrąglenia z powodu odejmowania bliskich sobie liczb lub dzielenia przez bardzo małą liczbę, to przybliżenie zaczyna odbiegać od dokładnej wartości pochodnej. Ponadto 2 ostatnie iteracje zwracają przybliżenie 0, ponieważ zgodnie z ekperymentami z zadania 3 w przedziale $[1, 2]$ krok wynosi 2^{-52} , więc dodanie liczb mniejszych od kroku nie zmienia wyniku i we wzorze otrzymujemy wtedy $\frac{f(x_0) - f(x_0)}{h} = \frac{0}{h} = 0$.